Analysis of BPU

余嘉閔, 113511060

Abstract—In this report, we analyze the efficacy of the current bpu in Aquila, including the ratio of it's hit: miss rate when bpu is in different sizes of BHT. And, we will also give an architecture of the tage predictor, which is a more efficiently bpu.

Keywords—BPU, BHT, Branch prediction

I. INTRODUCTION

BPU, a unit maintaining BHT, and is used to predict the next instruction when encountering a branch instruction in decode stage. It is crucial that we need to increase the efficiency of bpu, and decrease the rate of branch misprediction, in order not to flush the two instructions in fetch and decode stage and add nop to the pipeline frequently. To increase the BPU, we have to analyze the current bpu first, in this report I would analyze the ratio of it's hit: miss rate when bpu is different sizes of BHT, and the coremark score.

II. THE DATA OF THE CURRENT BPU

A. ENABLE AND DISABLE

BPU(size:64)	enable	disable
Score	105.6	94.6

Analysis:

- When BPU is enabled the coremark score raise by 11 Iterations/Sec, which is 11.4%.

B. The score of different BHT size and the hit : miss of Cond_branch and jal

BHT size		8	32	64	512
Score		102.3	105.1	105.6	106.4
Cond_branch	hit	6,632,433	10,585,148	12 226 579	14 368 684
	miss	6,567,502	3,511,261	1 734 350	13 971
	hit : miss	1 : 0.99	1 : 0.33	1 : 0.14	1 : 0.001
Jal	hit	0	950,073	1 162 980	1 257 401
	miss	326,069	2,898	13 044	13 375
	hit : miss	0 : 1	1 : 0.003	1 : 0.011	1 : 0.011

Analysis for the coremark score:

- The score gets larger as the BHT size become larger. The performance gain mainly results from the reduction of branch mispredictions, which lowers pipeline flush frequency and stall penalties.

Analysis for Conditional Branch Prediction:

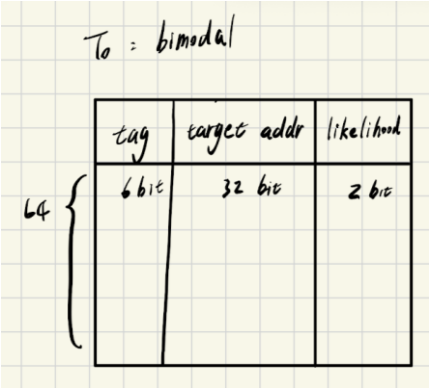
- The prediction accuracy increases dramatically as the BHT size grows from 8 to 512 entries, with the hit ratio improving from roughly 50% to nearly 100%.

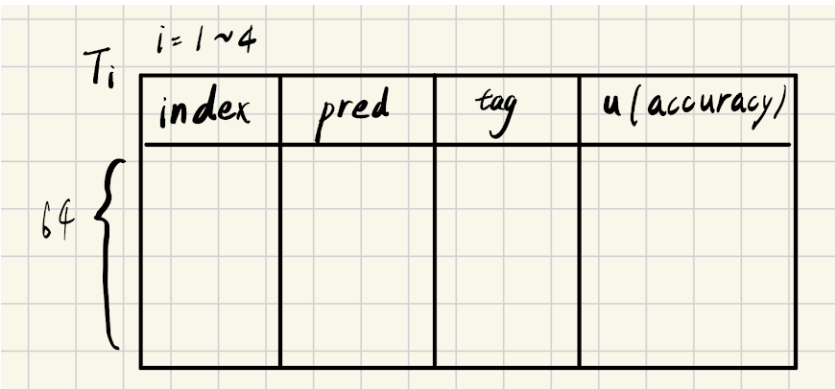
Analysis for JAL (Unconditional Jump) Prediction:

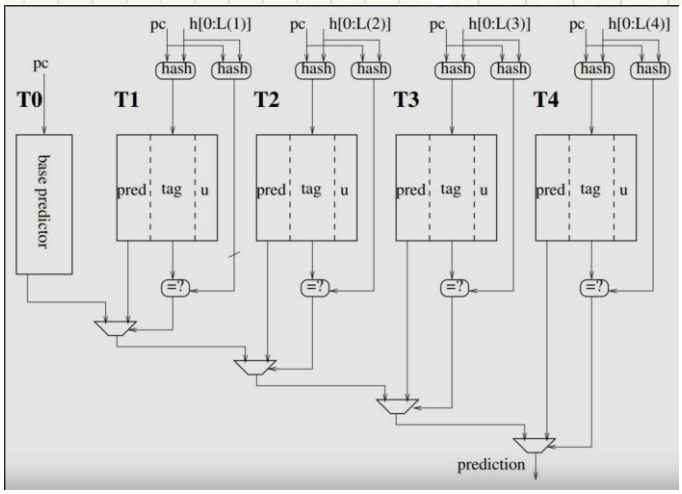
- With BHT = 8, all jump targets are mispredicted — the small table cannot retain any valid JAL history.
- Once expanded to 32 entries, the accuracy jumps to nearly 99.7%, showing that JAL prediction requires very limited history capacity.
- Beyond 64 entries, performance saturates; both hit and miss counts remain nearly constant.

Comparative Analysis: Conditional Branch vs. JAL:

- Conditional branches are the primary source of performance variation, as their prediction accuracy strongly depends on BHT size and aliasing effects.
- JAL predictions, by contrast, reach high accuracy with minimal resources.
- The major performance bottleneck arises from conditional branch mispredictions leading to pipeline flushes







III. ARCHITECTURE PLAN FOR IMPLEMENTING TAGE BPU

A. *description*

The BPU has five main blocks, T0~T4.

T0: same as bimodal one level bpu, it is used to maintain BHT and bimodal tag and likelihood, it acts like bimodal.

T1~T4: They are *Tagged tables, it only helps to decide whether to branch.(cannot impact the branch_hit)*

I use distri_ram to store it's tag(use index to find it), and reg array to store the likelihood(pred) and u(accuracy).

B. *How it operate:*

Three important varilables:

- History <= history<<1 + whether_branch_is_taken
- Index=pc^history
- Table_i tag = pc^(~history), the length is depend on i, and is exponentially increase along i(in my design tag_length t1:4, t2:8, t3:16, t4:32).

In fetch stage:

- It need to output branch_target_addr, branch_hit, decision. branch_target_addr, branch_hit is the same as bimodal. To find decision, we find the two longest tage table that is hit (if non of them hit, we can use bimodal's decision in that pc_addr). Then, we compare the u(accuracy) of the two table, the one with larger u to be our final decision(but if the larger u is still<3, we still choose bimodal because tage here is not accurate at all). Therefore, decision is it's likelihood.

In execution stage:

- *We have to maintain the bimodal BHT and tagged tabkes in this stage. Bimodal table is maintained the same way as 1 level does.*
- *To maintain tagged tables, first to consider whether to allocate a new entry in the next longer talke(if (no tage table hit or the longest two hit table both mispredict)& it hasn't have entry in table4 → allocate in the next longer tagged table)*
- *Next we have to maintain u(accuracy) and likelihood of that final_decision entry: if it predict correctly both u(accuracy) and likelihood+1, otherwise-1. But keep u in 0-6, likelihood in 0-7, if the operation will make it exceed the boundary, don't execute the operation. Besides, u is clear to 0 in a certain period of time.*

I. TAGE BPU RESULT ANANLYSIS

	default	**Simple hash algorithm**	**Tag1_length = 6**	**BHT size = 128**
Coremark score	105.308	103.987	105.295	105.523
Hit time	398,473,258	294,722,631	269,495,204	365,434,760
Miss time	54,648,909	54,649,903	54,649,428	50,538,949
Mispredict Time	12,870,275	19,139,916	13,083,729	14,299,425
Bimodal use Time	370,900,429	246,052,325	215,290,679	336,325,308
Tage use time	27,572,829	48,670,306	54,204,525	29,109,452
Bimodal wrong time	12,643,571	10,327,972	12,977,523	13,898,698
Tage wrong time	226,704	8,811,944	106,206	400,727
Tage predict correctly and it is a branch	26,259,619	33,740,789	52,955,079	27,586,708
Tage mispredict and it is a branch	1,690	499,198	42	258
Table1 mispredict time	226,704	8,811,944	106,206	400,727
Table 2-4 mispredict time	0	0	0	0
Hit : Miss	**7.29 : 1**	**5.39 : 1**	**4.93 : 1**	**7.23 : 1**
Tage accuracy	**99.18%**	**81.90%**	**99.80%**	**98.62%**
Bimodal accuracy	**96.59%**	**95.80%**	**93.97%**	**95.87%**
Use frequency (bimodal: tage)	**13.459**	**5.0549**	**3.973**	**11.556**
Mispredict : hit	**3.23%**	**6.50%**	**4.85%**	**3.91%**
(if tage mispredict) it's jump:not	**0.745%**	**5.663%**	**0.0395%**	**0.0644%**
(When tage is right) jump : not	**96.02%**	**84.64%**	**97.89%**	**96.10%**

Default:
complex hash algorithm, Tag1_length=4, BHT_size=64.

the measured data are as above.

Simple hash algorithm vs complex hash algorithm:

- I initially use the simple hash algorithm, but I found that all misprediction comes from table1, and it might because that hash is too simple that it hit easily in some special case (especially, it only has one kind of operation "xor"), so I revised table1 tag hash algorithm to make it more complex.
  - Simple: pc[L-1:0]^(~history[L-1:0])
  - Complex: pc[L-1:0]^(~history[L-1:0])+ history[L-1:0]
- complex hash algorithm has better efficacy.
- complex hash algorithm has less misprediction rate and higher accuracy and total hit rate(and it use tage bpu more frequently than complex one).

Tag1_length(the length of table1 tag): 4 vs 6 :

- similar coremark score.
- Length 4 has more hit rate,
- Length 6 has more accurate tage bpu.
- Length 6 is much tend to use tage bpu.(but I think it is strange)
- length 6 has to compare longer tag, therefore, it is less likely to hit but less wrong case is able to pass it, it rarely mispredict a branch instr that really jumps (with 0.0395%).

BHT size: 64 vs 128

- Size 128 has better efficiency(but not much difference).
- Size 128 has much higher misprediction rate, and it rarely mispredict a branch instr that really jumps (with 0.0644%).

Overall analysis:

- Hit rate > miss rate when BHT>64
- Tage accuracy is not positive relative to coremrak score.
  - Because bimodal accuracy and use frequency of the two bpu matters
- Tage rarely mispredict a branch instruction that really jumps, according to the last 2 row of data)
- Although, tage has higher accuracy than bimodal here, this two level bpu still has less coremark score than a pure bimodal bpu(score: 105.6).

What do I learn:

- Hardware design, especially those related to large system and those I still don't clearly understand, is greatly harder than software when it comes to debug. Therefore, it need more time to architect the architecture and detail before coding. And it is better that the design flow is already clear when starting to coding.